TAB HERO: TURNING SONGS INTO GAMEPLAY

Matthew Grohotolski
Susan Oseili
Chris Pleman

Introduction

Clone Hero [1] is a clone of the famous guitar simulation game Guitar Hero that involves pressing buttons on a guitar-shaped controller, where every button corresponds to a tab on the screen. Clone Hero currently only supports manually created charts, meaning every song must be individually transcribed to be playable. This process can be tedious and time consuming and transcription quality varies based on the skills of the transcriber. As a result, the song selection can be limited both in accessibility and scalability.

This project addresses this issue by creating a Deep Learning model that will automatically generate charts when inputted a raw audio file. Specifically, we aim to design a system capable of ingesting a song file and outputting a fully playable Clone Hero chart for all difficulties: Easy, Medium, Hard, and Expert. By automating the tab creation process, the system reduces reliance on manual transcription and expands the content of the game to include any song a user chooses. In addition, the model will also be able to identify song genres. This is important because the game expects a song's genre as input.

The motivation for this work stems from a shared enthusiasm for Clone Hero and music more broadly. As players of the game, we have experienced firsthand the limitations of song selection options and chart inaccuracies. By combining audio representation learning with sequence generation models and genre clustering, this work transforms a traditionally manual process into a scalable, data-driven system.

The supervised portion of this project frames chart generation as a sequence-to-sequence learning problem: given a raw audio file, the model produces a fully playable Clone Hero chart for a specified instrument and difficulty level. A single encoder-decoder transformer is trained on over 50,000 song-instrument-difficulty combinations, conditioning on both instrument identity and difficulty tier to cover all sixteen combinations from one model. After 66 epochs of training, the model achieved a best validation cross-entropy loss of 0.1085.

As an improvement upon similar works, the unsupervised portion of this project seeks to predict the genre of a song from its audio, frequency, and chart features. Three different clustering algorithms - KMeans, DBSCAN, and Gaussian Mixture Models (GMM) - are considered for their unique set of characteristics. Each model class is evaluated across several hyperparameter values to ensure that the best version of each algorithm can be compared against one another. The highest performing model was a GMM variant that achieved a silhouette score of .97, but only achieved homogeneity and completeness scores of .03 and .07, respectively, leading to poor clustering against validation data.

Data and Related Works

The closest example in this space includes Tensor Hero [2], another transformer based approach at automatically generating guitar tabs for the Clone Hero game. Tensor Hero generated Clone Hero tabs by feeding spectrographic data from parsed guitar audio files into a transformer. Without a publicly accessible dataset provided by this project, a unique dataset was generated from existing Clone Hero songs and converted into a custom .tab format, minimizing the chances of overlap. To expand upon this previous work we aimed to produce full-length instrument charts at all difficulty levels, a feature that is lacking in its predecessor.

Another example comes from a project that sought to automatically compose fingerstyle guitar tabs via a transformer model [3]. On its surface there is some overlap, however, this project is smaller in scope and its intended use differs from ours greatly. Our project seeks to produce chart data that is specifically formatted for a game, and attempts to do this for a wider instrument set: guitar, bass, rhythm and keys.

Kim, Yun and Kim is another related work that explores clustering music genres using signal derived features [4]. While their work focuses on both supervised and unsupervised learning, there is conceptual overlap in applying

PCA and KMeans to discover structure in music data. Their approach, however, is different as their data is derived from band features extracted from raw waveforms.

Feature Engineering

Preprocessing

The source of our dataset comes from ~6000 songs from the Clone Hero community with song-difficulty-instrument combinations. It includes additional metadata such as name, artist and genres. The preprocessing stage transforms raw song folders into a custom .tab binary file. This format stores preprocessed audio features and note sequences in a compact form optimized for training. The format is intentionally non-reconstructable -- the stored representations cannot recover the original source material.

The pipeline takes a raw song directory and uses HTDemucs [5] to separate the source into instrument stems such as guitar stems, bass stems and piano stems. The audio is then stored as a Mel Spectrogram, which is a lossy transform that discards phase information and therefore cannot be converted into audio, preserving the patterns without copying the original work. Charts are then parsed from the .chart or .mid source file, game mechanics such as Star Power and hammer-on pull-off (HOPO) flags are decoded, and the full representation is packed into a custom binary .tab format. Each .tab file stores the log-Mel spectrogram and tokenized note sequence together in a compact form that is significantly faster to load during training than raw audio files.

Because full songs can span thousands of mel frames and tens of thousands of note tokens, training operates on chunked windows rather than complete songs. The ChunkedTabDataset partitions each .tab file into overlapping windows with a configurable frame overlap to maintain cross-boundary context. SpecAugment is applied on-the-fly during training: time and frequency masking blocks are randomly applied to mel spectrogram chunks, preventing the model from overfitting to specific spectral patterns.

Statistical Exploration

All numeric features were standardized using z-score normalization prior to PCA, KMeans clustering and nearest neighbor analysis. Centering each feature at zero mean and unit variance ensured proportional contribution across distance and variance based methods.

After converting raw Clone Hero directories into standardized .tab format, per chart statistical features were extracted to evaluate structural relevance, stability and redundancy for supervised tab generation and unsupervised clustering. Each .tab file contains a Mel Spectrogram and tokenized note sequence. The spectrogram was converted from log to linear scale for energy computations. Tokens were decoded via ChartTokenizer.decode_tokens to recover NoteEvent objects containing timestamps, fret combinations, durations and modifiers.

Observed feature groups included:

Density and timing: notes_per_second_mean, n_notes, inter_note_ms_mean, inter_note_ms_std, inter_note_ms_median, inter_note_ms_p25, inter_note_ms_p75, duration_sec

Structural gameplay: chord_ratio, sustain_ratio, fret_entropy, hopo_ratio, tap_ratio, star_power_ratio

Audio descriptors: mel_rms_mean, mel_rms_std, rms_energy_mean, rms_energy_std, spectral_centroid_mean, tempo_bpm

Difficulty validation showed strong monotonic structure. Notes_per_second_mean increased from Easy to Expert, while inter_note_ms_mean decreased. Kolmogorov Smirnov (KS) distance matrices confirmed substantial distributional separation, particularly for density and timing features. Gameplay ratios exhibited threshold effects rather than continuous scaling. For example, chord_ratio separated Easy from higher tiers but plateaued thereafter. Spearman correlations confirmed strong monotonic associations. Approximately 94.2 percent of charts were nearest to adjacent difficulty tiers, indicating a smooth ordinal manifold. Tempo_bpm showed negligible structural relevance.

Instrument level analysis revealed consistent ordinal progression within instruments, but substantial cross-instrument drift in absolute density and mechanical complexity. Lead and Rhythm charts showed higher density ceilings, while Keys exhibited lower density and higher sustain ratios. KS matrices quantified this instrument conditioned shift, stressing the importance of instrument_id for modeling.

Intra difficulty clustering demonstrated moderate internal substructure. Silhouette scores indicated consistent but overlapping partitions, and centroid analysis showed that clusters different primarily along density and timing gradients. In conclusion, difficulty labels define density bands rather than unique structural styles. This aligns with the underlying goal of tab generations - difficulty levels reflect the same songs and therefore the same structure.

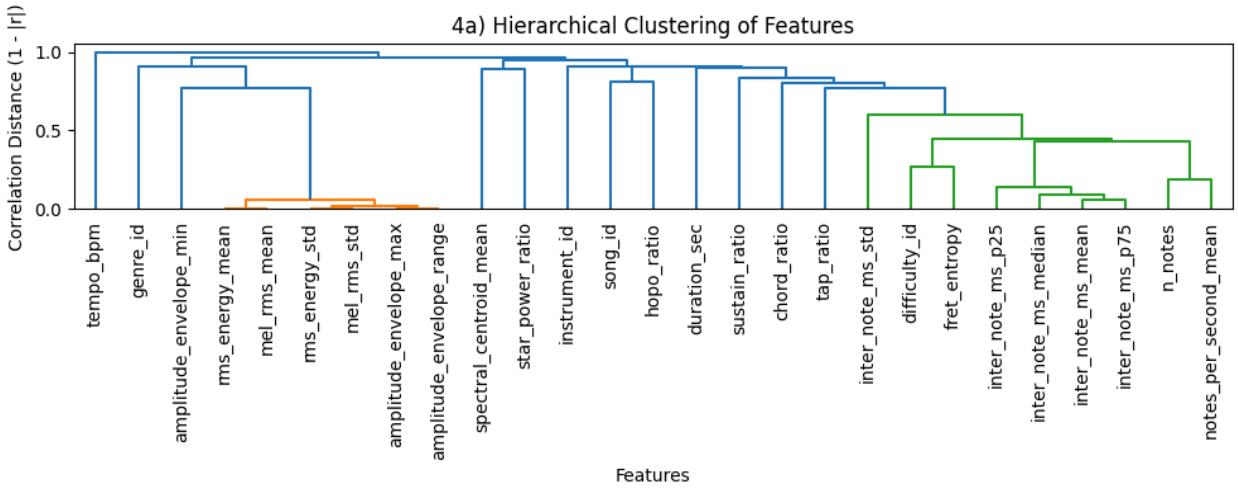

Figure 1: Dendogram showing the hierarchical clustering of features based off of the cleaned rows, calculating using the Pearson correlation coefficient with a distance metric of 1 - |r| applied to achieve a scale of 0 to 1, with 1 indicating a strong relationship and 0 indicating no relationship.

Correlation clustering and PCA revealed strong redundancy in the data. Density and timing metrics formed a tight block, while RMS based audio features created a separate cluster. The first principal component was dominated by density and timing metrics. Drop-one-feature analysis showed notes_per_second_mean drastically affected cluster assignments, whereas tempo_bpm had little to no effect. Bootstrap resampling confirmed structural feature stability and higher variability in some dispersion and RMS metrics. Redundant or uninformative variables included tempo_bom, mel_rms_std, inter_note_ms_mean, amplitude envelope statistics and n_notes.

Genre analysis showed weaker structure. ANOVA F-scores identified spectral_centroid_mean as the strongest discriminator, with modest contribution from density and chord metrics. PCA projections showed overlapping but shifted genre distributions, confirming the weak separation between genre lines.

The final feature set is retained for unsupervised learning notes_per_second_mean, chord_ratio, sustain_ratio, spectral_centroid_mean, and rms_energy_mean to reduce redundancy and improve interoperability. For supervised learning, the most important data included instrument_id and difficulty_id.

A row level cleaning stage was then applied. This stage enforced duration consistency within songs, removed charts with suspiciously low note counts and corrected near zero fret_entropy artifacts. All analyses were recomputed post cleaning to compare results as a sanity check.

Cleaning reduced artificial variance in timing and density metrics while preserving ordinal difficulty structure. Approximately 91.9 percent of charts remained nearest to adjacent difficulty tiers. Correlation structure remained consistent, though several magnitudes decreased slightly which indicated a reduction in noise. PCA also shifted toward a more balanced representation of structural and audio variance, giving insight into the impact of the low duration and low note songs on variance. Clustering stability improved after cleaning. Silhouette scores remained

moderate, but centroids were less influenced by the large number of charts with skewed timing, giving more balanced results. Features such as notes_per_second_mean remained influential while tempo_bpm remained negligible. Instrument conditioned drift also persisted with a more stable effect size. Genre analysis remained largely unchanged.

Overall, the data cleanup improved statistical coherence without altering substantive conclusions. Density and timing metrics remain the primary drivers of difficulty progression. Mechanical ratios define structure. Instrument conditioned differences are substantial, while genre effects are comparatively modest.

Supervised Learning

For supervised training, the note event sequence is tokenized using a ChartTokenizer that encodes each note as exactly four consecutive tokens: a time delta token quantized to 10ms bins, a fret combination token encoding the active fret bitmask as a single integer index, a modifier token capturing HOPO, tap, and star power flags, and a duration token quantized to 50ms bins. This scheme produces a vocabulary of 740 tokens including special tokens for beginning-of-sequence, end-of-sequence, and padding. Conditioning information for instrument identity and difficulty level is injected as learned embeddings in the decoder rather than as tokens, keeping the sequence length determined purely by note count.

Chart generation is formulated as a supervised sequence-to-sequence learning problem: given a log-Mel spectrogram derived from an audio file, the model produces a sequence of note tokens that form a playable Clone Hero chart at a specified instrument and difficulty level. The learning method is a causal encoder-decoder transformer [6] selected for its established strength in long-range sequence modeling and its native support for cross-attention between audio and token modalities. The architecture is described across three functionally distinct supervised components.

The audio encoder consists of a shallow convolutional front-end that projects 128-band log-Mel spectrogram frames into a 768-dimensional feature representation. A 4x temporal downsampling is applied before the encoder output is passed to the decoder through cross-attention, reducing the sequence length the decoder must attend over while retaining spectral information at coarser time resolution. This downsampling also reduces VRAM requirements, which is critical given that songs can span many thousands of frames.

The sequence decoder is an 8-layer causal transformer with 12 attention heads and a feed-forward dimension of 3072. Rotary Positional Encoding (RoPE) [7] is used in place of conventional fixed position encodings because RoPE encodes the relative distance between positions rather than each token's fixed index, which helps the model generalize to sequences longer than those seen during training. Flash Attention 2 [8] is used for memory-efficient attention computation. Two learned conditioning embeddings, one for instrument identity (lead, bass, rhythm, keys) and one for difficulty tier (Easy, Medium, Hard, Expert), are added to the decoder input sequence. This allows a single trained model to produce outputs appropriate for each of the sixteen instrument-difficulty combinations without separate model heads. The full model contains approximately 100 million parameters (99.8 million non-embedding).

At inference, constrained decoding is applied by restricting the output probabilities at each step to only the valid token type for that position in the quad-token cycle: TIME -> FRET -> MODIFIER -> DURATION, repeating. This mask is applied before sampling and prevents the generation of structurally invalid sequences such as back-to-back time tokens or notes with impossible fret combinations. Sampling uses top-k filtering (k=50) followed by nucleus sampling (p=0.95) [9], providing a balance between determinism and variety.

The model is trained using cross-entropy loss over the full token sequence, computed at each of the four positions within each note quad, with the ground-truth previous token fed as input at each step. The optimizer is AdamW [10] with beta values of 0.9 and 0.95, weight decay of 0.01, and a base learning rate of 5e-5. Learning rate follows a cosine decay schedule with a 1000-step linear warmup. Training uses BF16 mixed precision and gradient accumulation over 4 steps, yielding an effective batch size of 28. Gradient norms are clipped at 1.0. Training ran on an RTX 5090 with 32GB VRAM. The training set consisted of approximately 50,714 preprocessed .tab files and the validation set consisted of 1,192 held-out files. Maximum context was 16,384 mel frames per chunk with 1,024-frame overlap. Training was configured for up to 100 epochs with early stopping at patience of 15; the final run completed 66 epochs. The top 5 checkpoints by validation loss were retained throughout.

Cross-entropy loss was selected as the primary evaluation metric because chart generation is fundamentally a token-level prediction task at each decoding step, and cross-entropy directly measures the model's per-token prediction confidence on held-out data. Downstream playability metrics such as note onset accuracy would be more interpretable, but require access to ground-truth audio-aligned chart renderings or human raters that were not available at scale. Cross-entropy on a held-out validation set provides a consistent, reproducible measure of generalization that is directly aligned with what the model was trained to optimize.

Figure 2 shows the training and validation loss curves across all 66 epochs. Training loss began at approximately 6.85 at epoch 0, reflecting near-random token prediction from an untrained model (the theoretical maximum for a uniform distribution over 740 tokens is approximately 6.6 nats), and decreased steadily to approximately 0.29 by epoch 65. Validation loss, first recorded at epoch 33 at 0.1718, declined monotonically through epoch 65 where it reached a minimum of 0.1085. The gap between training loss and validation loss reflects the fact that validation was computed on full-length chunks without SpecAugment [11], while training loss was computed on augmented chunks under teacher forcing. The absence of divergence between training and validation loss curves through epoch 65 indicates that the model was not in a regime of severe overfitting at the time training concluded.

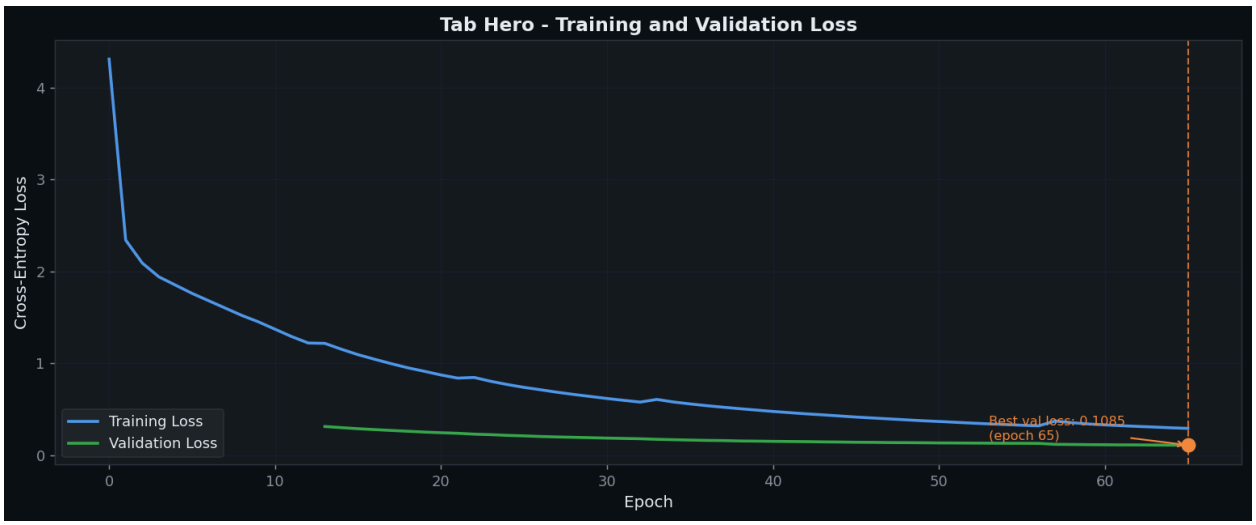

Figure 2: Training and validation cross-entropy loss across 66 epochs of training. Validation loss reached a minimum of 0.1085 at epoch 65.

Figure 3 shows an example piano-roll visualization of a chart generated by the trained model for a sample audio track at Expert difficulty, bass instrument, using temperature 1.0, top-k 50, and top-p 0.95 sampling. The chart spans approximately 60 seconds and demonstrates the expected density and fret distribution characteristics of an expert-difficulty bass chart.

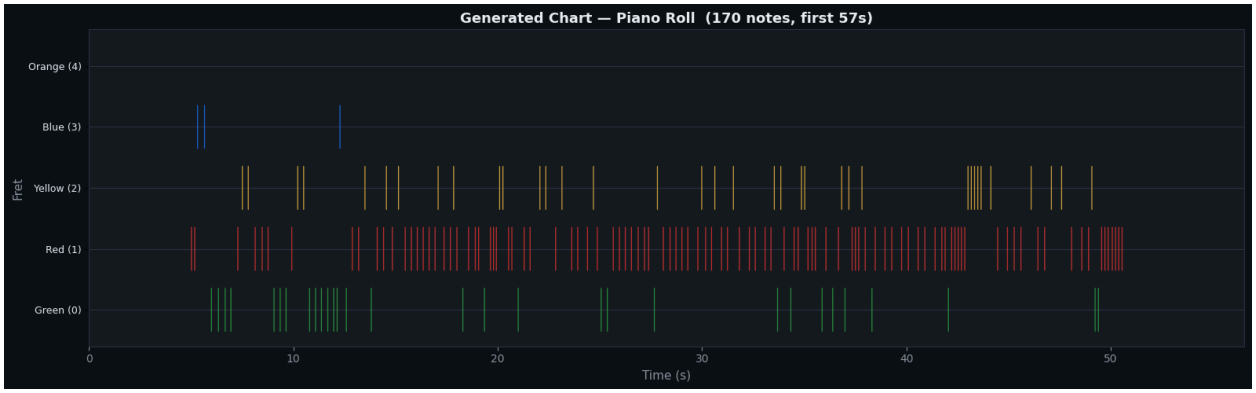

Figure 3: Generated piano-roll chart from inference for bass at Expert difficulty. Fret lanes: Green=0, Red=1, Yellow=2, Blue=3, Orange=4. Rectangle width represents note duration.

Among the design choices, the most impactful feature for supervised chart generation is the Mel spectrogram representation itself. The 128-band log-Mel spectrogram provides a compact and musically meaningful encoding of the audio signal, capturing the spectral patterns associated with note onsets, rhythmic structure, and instrument timbre. Without the audio input, the decoder would have no acoustic grounding and could only model average difficulty-level statistics.

The instrument and difficulty conditioning embeddings are the second most impactful features. If these conditioning signals are omitted, the model collapses toward generating a single average chart pattern regardless of the requested difficulty or instrument. Difficulty conditioning directly corresponds to the density and complexity gradients identified in the feature engineering analysis: Expert charts require far more notes per second and more complex fret patterns than Easy charts. Without difficulty conditioning the generated chart does not reflect these structural differences. Without instrument conditioning, the generated output fails to capture the distinct structural ceilings and stylistic norms observed across instruments in the exploratory analysis, particularly the lower note density ceiling and higher sustain ratio of Keys relative to Lead.

RoPE positional encoding is significant for handling variable-length songs. Conventional fixed position encodings break down when sequences are longer than those seen during training, producing garbled outputs toward the end of long songs. RoPE avoids this by encoding the relative distance between positions rather than their fixed location in the sequence.

The 4x audio downsampling in the encoder represents a design tradeoff with meaningful quality impact. At 2x downsampling, training was computationally prohibitive for long-context chunks. At 8x downsampling, fine-grained onset information was lost, resulting in noticeably worse note timing precision. The 4x factor represents a practical balance between temporal resolution and training efficiency.

The generation output is most sensitive to inference temperature. At temperature 0.7, the model produces structurally coherent charts with high repeatability but reduced stylistic variety, often defaulting to common fret patterns at the expense of rhythmic variation. At temperature 1.0, output diversity increases substantially with no meaningful increase in structural error rate due to constrained decoding. At temperature 1.2 and above, structural error rate increased and the constrained decoding had to discard more proposed tokens, sometimes producing implausible timing sequences. A temperature of 1.0 was selected as the inference default.

The cosine learning rate schedule with linear warmup proved critical for training stability. Runs without the warmup phase diverged within the first 100 steps, producing NaN loss values. Runs with warmup converged smoothly. The 1,000-step warmup corresponds to approximately one-half epoch given the dataset size, and allowed the model to stabilize before the full learning rate was applied.

Three primary tradeoffs were identified during evaluation. First, generation quality versus inference speed: the 100M parameter model generates charts approximately in real-time on a modern GPU but would be prohibitively slow for CPU deployment. A smaller model variant with 256-dimensional encoder and decoder, 4 decoder layers, and approximately 15M parameters produced noticeably lower-quality charts but generated at roughly 10x the speed. The large model was used for all reported results.

Second, temperature versus structural validity: higher temperatures increase note variety but push the model toward tokens that constrained decoding must reject, occasionally producing rhythmically awkward sequences in rapid passages. The tradeoff between musical creativity and structural fidelity is not fully resolved by constrained decoding alone, particularly in dense Expert sections.

Third, training context length versus validation score: training on 16,384-frame chunks versus 8,192-frame chunks improved validation loss by approximately 0.01 nats but doubled VRAM requirements, requiring a corresponding reduction in batch size. The longer-context run achieved the reported best validation loss of 0.1085.

Three representative failure modes were identified during evaluation, see below:

- **Example 1 - Timing compression in dense solos**
    - In high-tempo Expert lead charts, particularly in guitar solo passages where notes per second exceeds 15, the model tends to underestimate time deltas and generates tight clusters of notes at or near the minimum quantization bin of 10ms. The resulting chart contains a burst of notes that are physically unplayable in sequence. This failure is attributed to insufficient representation of dense passages in the training data relative to normal-density sections, and to the coarse 10ms time resolution limiting fine-grained timing control in rapid passages.

- **Example 2 - Sustain duration underestimation**
    - For instruments and difficulty levels where long sustains are common, particularly Keys charts at lower difficulties, the model consistently generates shorter duration tokens than the ground truth. Duration tokens in the short-note range are overrepresented in training because most chart styles involve rapid notes, and the model has developed a prior toward short durations. The result is a Keys chart that resembles a Lead chart with rapid plucking rather than held chords, failing to capture the characteristic sustain-heavy structure identified in the feature engineering analysis.

- **Example 3 - Context drift in long songs**
    - For songs exceeding approximately 4 minutes, the generated chart in the final 60 to 90 seconds begins to diverge from the stylistic pattern established in earlier sections. Note density increases irregularly, fret patterns become more repetitive, and the difficulty level no longer tracks the actual musical intensity of the audio. This is attributed to the finite context window of the chunked inference pipeline and the loss of long-range musical context across chunk boundaries. Although streaming inference uses a 512-frame overlap between chunks to maintain short-range continuity, this is insufficient for large-scale musical structure.

Future improvements that could address these failures include augmenting the training set with oversampled examples of dense and sustain-heavy passages, replacing fixed quantization bins with variable-resolution token schemes for time and duration, and implementing a hierarchical structure model that maintains song-level context information across inference chunks.

The primary finding of the supervised learning section is that a sequence-to-sequence transformer trained with teacher forcing on a tokenized chart representation can generate structurally plausible and musically coherent Clone Hero charts from raw audio. The validation loss of 0.1085 after 66 epochs is substantially lower than the theoretical random baseline of approximately 6.6, indicating genuine learning of note timing, fret pattern, and difficulty structure from the mel spectrogram input.

A somewhat surprising result was that the instrument and difficulty conditioning worked as effectively as it did given that only learned embedding vectors were used for conditioning, rather than more explicit feature injection or separate model heads. The single unified model handles all 16 instrument-difficulty combinations without noticeable output collapse or mode mixing, suggesting that the conditioning embeddings successfully capture the structural differences identified in the feature engineering analysis, particularly the density and complexity gradients across difficulty tiers and the note-density and sustain-ratio differences across instruments.

The most significant challenge was dataset preprocessing: converting approximately 6,000 song directories into the binary .tab format required implementing source separation, mel spectrogram extraction, and chart tokenization in a single reproducible pipeline. Data quality was variable across the community-sourced Clone Hero charts, and several hundred songs were excluded during row-level validation due to duration inconsistencies and implausibly low note counts.

With more time and resources, the supervised system could be extended in several meaningful directions. A human preference dataset of generated versus manually-transcribed charts could enable reinforcement learning from human feedback to optimize directly for playability rather than cross-entropy loss. Beat-aligned decoding, which constrains note onsets to musically meaningful positions such as beat boundaries or eighth notes, would likely reduce the timing quantization errors observed in dense passages. Multi-instrument coherence constraints ensuring that

simultaneously generated Lead and Bass charts share consistent rhythmic structure would produce more realistic full-band chart sets.

Unsupervised Learning

Musical genres are classification of is a categorization of music based on shared form and style [12]. This is illustrated by the clear differences in tempo, loudness, instrument types (and more) between common genres such as Rock, Classical, or Country songs. Most listeners can reasonably know, just by listening to the song, which genre it belongs to. Using this idea of musical genre, we aim to design a clustering algorithm that can mimic our ability to classify genre. To achieve this, we need to generate features that are representative of these musical characteristics that a machine learning algorithm can process.

These types of features can fall into two key domains: the time-domain, referring to the amplitude of the waveform over time, and the frequency-domain, referring to the actual sound frequencies captured in the audio file. Relevant time-domain features for music include the Root Mean Square (RMS) energy and the Amplitude Envelope (AE). The RMS represents the average power, or intensity, of the audio signal over a given period of time, which could meaningfully differentiate between genres that are louder or quieter than others (e.g., Reggae vs Heavy Metal). The AE, on the other hand, represents the general shape of the waveform by tracing the crest of the waveform over time. This type of feature could help differentiate between more and less consistent music - compare the constant kick drum of a Rock song to the irregular form of Jazz.

The primary frequency-domain feature includes the Spectral Centroid Mean (SCM) which represents the average brightness of a sound. This type of feature could help differentiate between some Pop music, which is generally brighter, and some Classical music which can be darker and more dramatic.

For the purposes of detection genre via an unsupervised learning algorithm, the KMeans, DBSCAN, and GMMs methods were used. Each of these algorithms have unique benefits that could be valuable for predicting genre:

- The KMeans algorithm is fast, can predict new labels after training, but requires the number of predicted clusters to be predetermined.
- The DBSCAN model does not require that the number of clusters be predetermined, can detect non-spherical clusters, but does not natively predict new clusters after training.
- The GMM can predict new labels after training, but requires the number of mixture components to be predefined, and assumes that the dataset is a mixture of normally distributed clusters.

The dataset used for clustering was based on the features.csv file that was a collection of extracted audio, frequency, and chart features for each instrument and difficulty level. This included features like Root Mean Squared Energy, Mean Spectral Centroid, Notes per second and more [13].

However, the dataset was too granular and in the incorrect format for clustering songs into genres. For instance, a single song could have up to sixteen records each, composed of four instruments with four separate difficulty levels. To manage the duplication caused by multiple difficulty levels, only the highest difficulty level was kept for each instrument. This option was chosen because the highest difficulty level was considered to be the most accurate representation of the original song. Moreover, averaging these metrics would not be appropriate, because the resulting chart feature values would have been artificially modified by the lower difficulty level values.

Although representation at the instrument level was handled, the dataset needed to be pivoted to accurately perform genre classification. Within the context of this project, a song is the composition of several instruments playing concurrently, but this relationship would be erased if each instrument was spread across rows. Moreover, if a song does not have a particular instrument, that could be valuable information that would be lost in this long format. Additionally, this format could cause the models (particularly DBSCAN) to cluster on song-level features that would necessarily be the same across the instrument records.

Despite solving the formatting problem, pivoting presented an issue with when and how to handle null values and normalization. The solution was guided by the need to standardize the new dataset, while retaining the meaning that the absence of an instrument implied. Both the MinMaxScaler and the StandardScaler were tested against the dataset. The MinMaxScaler was initially considered for its restricted return value range of [0,1], allowing imputation

with a value that would not be possible if an instrument record was present. Imputing with a value of -1 after normalization was meant to retain this meaning while retaining the general range of the MinMaxScaler. On the other hand, the StandardScaler returns the z-score for each record - a function with an infinite range. To manage this, each feature was imputed with a negative value twice as large in magnitude as its most extreme value. This dynamic imputation was selected over a single arbitrary value to better reflect the standardized distribution of each feature in the dataset. Ultimately, the StandardScaler was used as all 10 principal components derived from applying PCA to its standardized dataset captured more of the variance in the data than that of the MinMaxScaler dataset (Figure 4).

The application of PCA also served to address the curse of dimensionality. Pivoting the data quadrupled the number of features while simultaneously reducing the dataset size to just under 6,500 records. Through the application of PCA, the feature size could be reduced to three principal components while retaining 98% of the variance in the dataset (Figure 4).

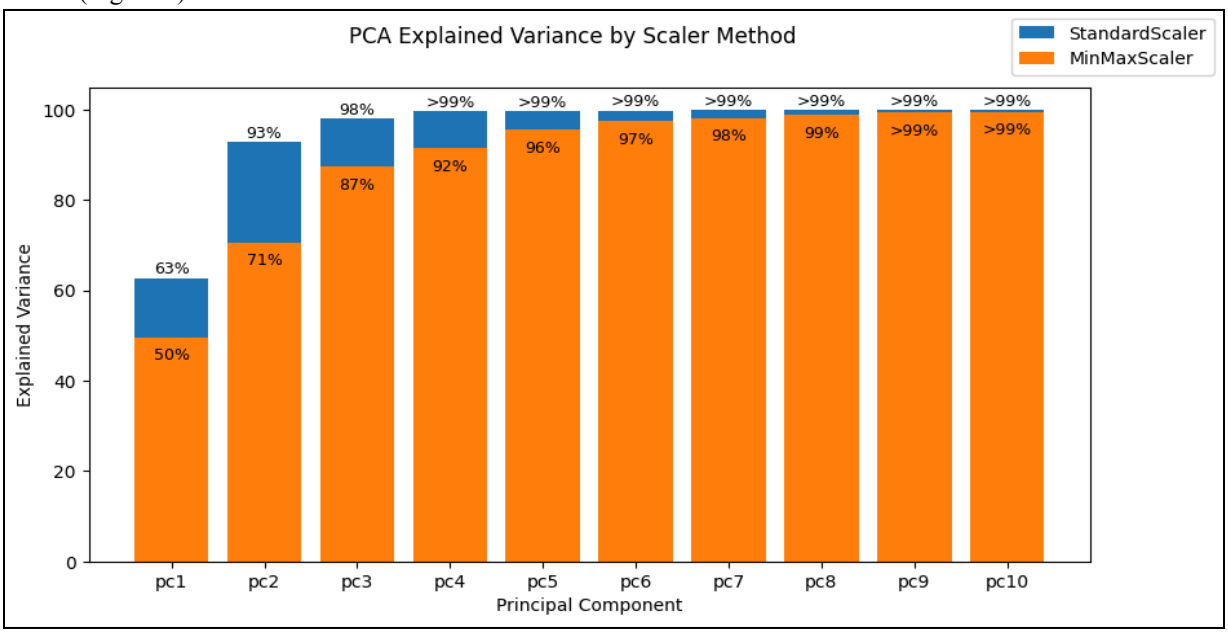

Figure 4: Explained variance ratio for PCA components based on each scaler method.

To evaluate the performance of each of the three models, three metrics were used: the silhouette, homogeneity, and completeness scores. The silhouette score measures how close points in one cluster are to other clusters, with high values representing good separation between clusters - but have no guarantee the identified clusters accurately represent the "true" clusters. However, the unsupervised dataset did contain true genre labels, therefore the homogeneity and completeness scores were used. These metrics directly compare predicted labels to the true labels, where a high homogeneity score reflects clusters composed of only one true cluster, while a high completeness score represents true clusters assigned to one predicted cluster. For this reason, when the number of predicted clusters is less than the true number of clusters, completeness should be higher than homogeneity, and the opposite is true when the number of predicted clusters is greater than the number of true clusters.

Due to the drastic imbalance in the distribution of the genres (Figure 10) and KMeans' difficulty with handling datasets of this nature, the n_clusters hyperparameter was tested across values of 8-24 (5-1.5x the number of actual genres) to allow for the possibility of detecting super- and sub-genres. However, increasing the number of clusters decreased the performance of the silhouette and completeness scores, while only slightly increasing the homogeneity score. The max_iter hyperparameter was also tested with values ranging from 200-400, but this had no impact on model performance (Figure 5).

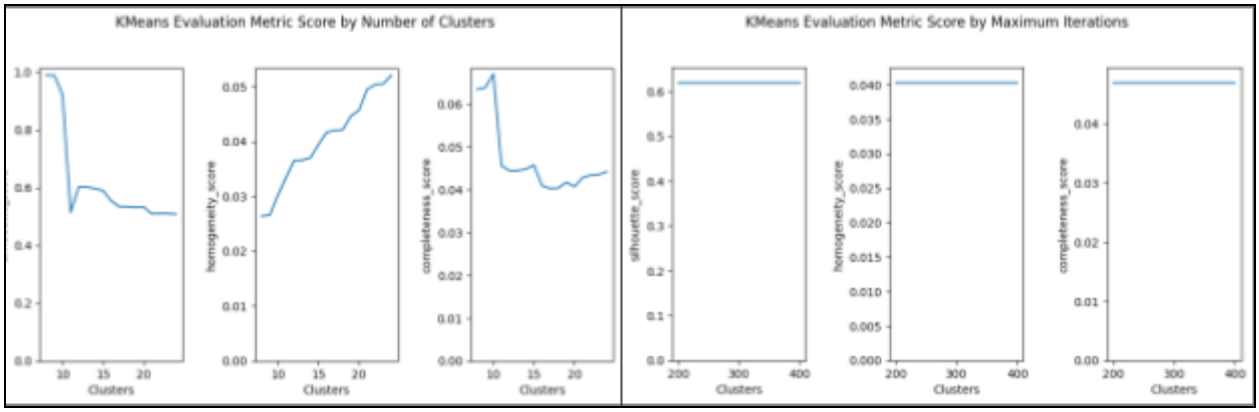

Figure 5: Hyperparameter tuning results for the KMeans algorithm across the three evaluation metrics.

The DBSCAN model performance was evaluated by varying the values of the eps and min_samples hyperparameters due to their relative impact on the algorithm's output. The eps hyperparameter was tested across a range of .25-.75 (steps of .05), because performance was most sensitive within this range. This hyperparameter had varying effects on model performance across all three metrics, however the highest general performance occurs between eps values of .5-.6. Additionally, the min_samples hyperparameter was tested over a range of 1-9 to test a wide range of possible cluster densities for the model to identify, with a value 2-3 returning the highest general performance across each metric (Figure 6).

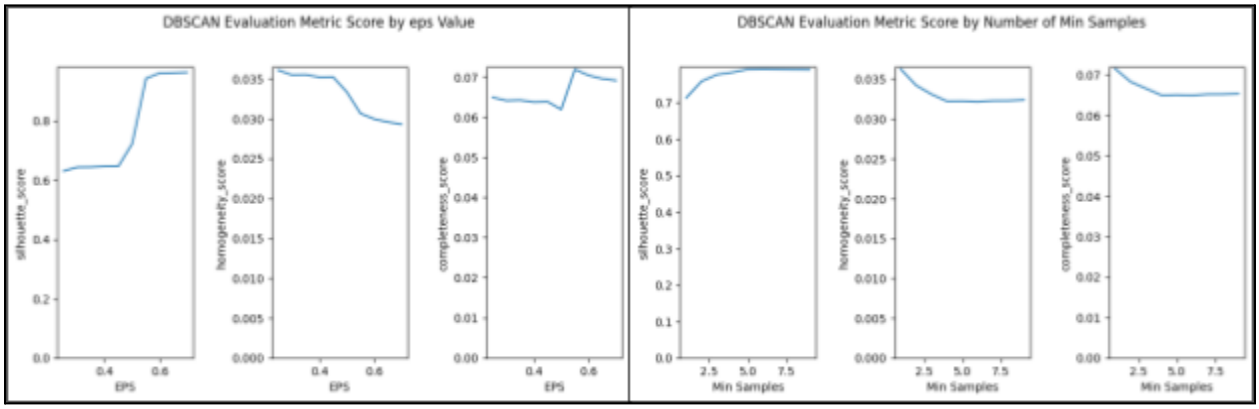

Figure 6: Hyperparameter tuning results for the DBSCAN algorithm across the three evaluation metrics.

Lastly, the GMM was tested by varying its n_components (number of mixture components) and covariance_type hyperparameters. The former was tested across the same range as the KMeans model for consistency, and the latter was tested across each of the four covariance types: 'full', 'diag', 'spherical', and 'tied'. Changing the number of mixture components affected the GMM's performance in broadly the same way the n_clusters affected the KMeans algorithm's performance. The different values of the covariance_type had a relatively moderate effect on the silhouette score, and a minimal effect on the homogeneity and completeness scores (Figure 7).

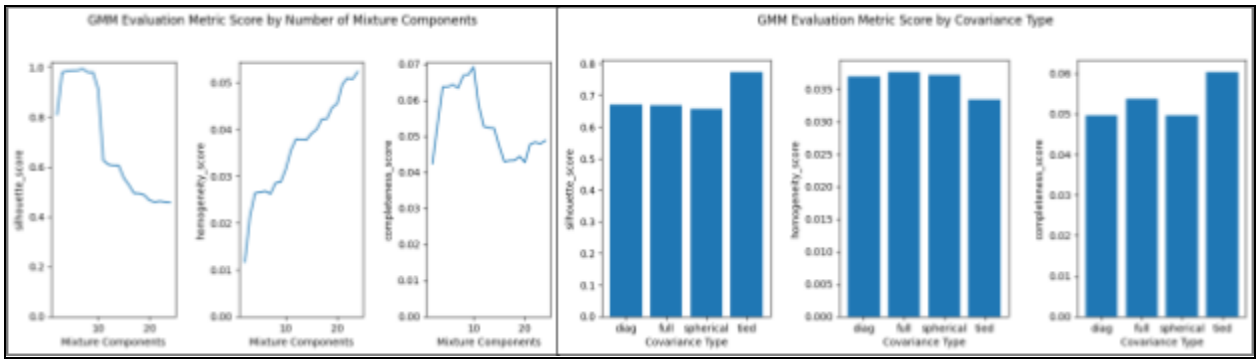

Figure 7: Hyperparameter tuning results for the Gaussian Mixture Model across the three evaluation metrics.

Model selection involved comparing the best variant across each model class. The 'best' variant within a model class was identified by having the highest performance across all three metrics simultaneously. This was achieved by normalizing the performance metrics with the MinMaxScaler and computing the average of the normalized metrics. However, due to how poorly every model performed on the homogeneity and completeness metrics, the silhouette scores were given slight preference by culling any models that produced negative silhouette scores. The variant with the highest performance in this composite metric was compared against the top variant in the other model classes (Figure 8).

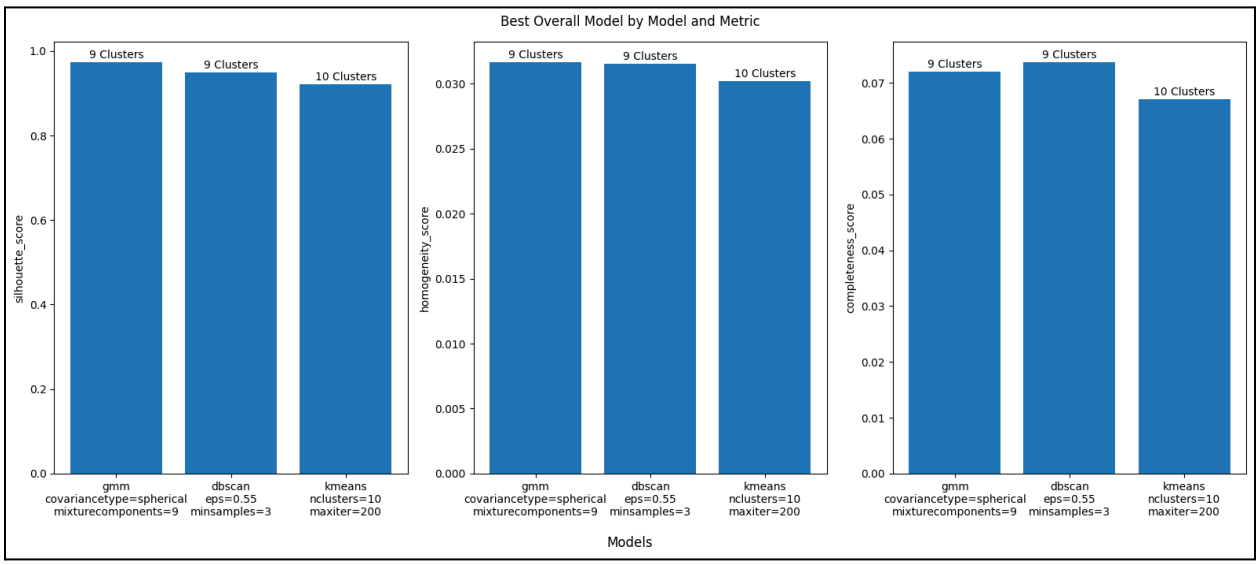

Figure 8: 'Best overall' model variants as an average of normalized metric scores. Any model with raw silhouette scores were removed from consideration.

Ultimately the best model variant was the GMM with a spherical covariance type and 9 mixture components. This variant achieved a silhouette score of .97, but only achieved homogeneity and completeness scores of .03 and .07, respectively. Although it did not reliably cluster the data into the expected number and types of genres, comparing the predicted and true clusters for the first two principal components provides helpful context for this performance (Figure 9).

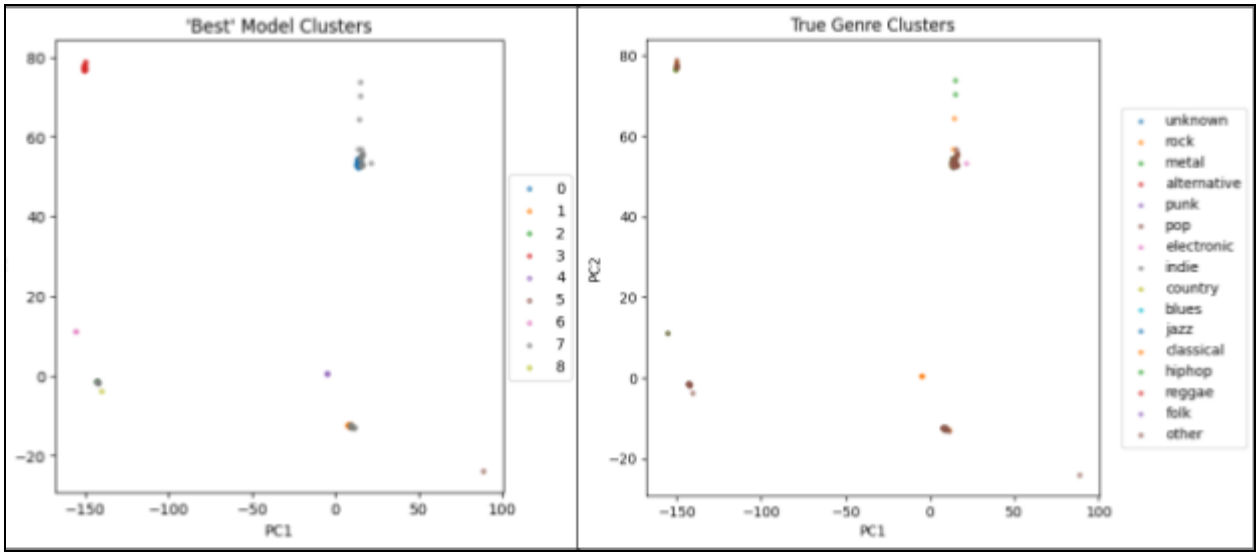

Figure 9: Side-by-side comparison between the predicted labels by the best model variant and the true genre clusters and their labels.

Discussion

Supervised Learning

When we started the supervised portion of this project, the assumption was that generating a playable chart from audio would require a lot of hand-holding: a note detection model, a beat tracker, difficulty filters, and post-processing layered on top of one another. What we found is that if you give the model enough examples of how good charts look alongside the audio they correspond to, it figures most of that out itself. The architecture matters, but the data does more of the work than we expected going in.

The result that caught us most off guard was how well the conditioning held up across all sixteen instrument-difficulty combinations. We were prepared for output collapse in underrepresented combinations and had planned contingencies around separate model heads if it happened. Neither was needed. In retrospect, it makes sense: the structural differences between instruments and difficulties are already so visible in the training data that the model had plenty of signals to work from. The challenge was not getting it to differentiate; it was making sure the training set was clean enough to teach it at all.

That brings us to what actually took the most time: the data pipeline. Converting roughly 6,000 raw song directories into training-ready files required building source separation, spectrogram extraction, and chart tokenization into one end-to-end process, and the quality variation in community charts made it harder than expected. As players we already knew charts varied widely. What we did not anticipate was how many records would need to be excluded, and how much those early cleaning decisions would ripple through to final model performance.

Given more time and resources, the failure modes identified in evaluation point to the most valuable next steps. Oversampling dense and sustain-heavy sections during training would address timing compression and sustain underestimation directly, without requiring architectural changes. Adding beat-aligned decoding would anchor generated notes to musically meaningful positions and help most in the dense Expert passages where the model struggles. The extension we are most interested in longer term is a human preference stage where players rate generated charts against hand-authored ones and the model is fine-tuned toward higher-rated outputs. Optimizing for what a chart feels like to play, rather than prediction accuracy alone, is where the largest quality gap remains.

Unsupervised Learning

Overall the models performed poorly at clustering songs into their true genre groups. This was particularly surprising given that the most impactful features from the features.csv file were used, and the principal component transformation managed to capture much of the variance in that subset of data.The level of poor performance makes it difficult to meaningfully evaluate the clusters. Although the homogeneity and completeness scores were originally regarded as the most important metrics, every model produced such low scores that comparing them was not meaningful. As a result, greater importance was applied to the silhouette score, as it had the most variation and high cluster separability is still a valuable metric.

Ultimately this suggests that the dataset was deficient in its scope, contained generic features, or both. For instance, utilizing a metric like rms_energy_mean could drive all of the variation towards a central value. This feature representation could have erased the important variation required to differentiate songs by genre. Similarly, using only the RMS and SCM likely did not capture enough of the descriptive characteristics of a song to effectively perform this task. With more time and resources, investigating and generating a broader set of descriptive features would be wise. Additionally, addressing the extreme genre imbalance through obtaining more data for each genre type would also be essential. Lastly, including lyrical analysis could be a valuable additional feature set for more robust genre detection.

Statement of Work

Susan Oseili was responsible for the exploratory data analysis and feature engineering portion of the project. She created notebooks first exploring the .tab files and the features.csv before doing a further deepdive creating visualizations and results analysis for the full features dataset to discover appropriate features to use for supervised and unsupervised learning. She consolidated some shared logic between the exploratory data analysis and unsupervised learning.

Matthew Grohotolski was responsible for creating the data preprocessing pipeline that ingested raw audio files and generated the .tab files as well as the tokenization to populate the dataset that was utilized by the whole team. In addition, he was responsible for the supervised portion of the project, creating a PyTorch Transformer for the pipeline as well as utilizing the RoPE encoder. He also created supporting visualizations for this task.

Chris Pleman was responsible for the unsupervised portion of the project, investigating KMeans clustering, Gaussian Mixture Models and DBSCAN and ranking model performance against one another. To support his work, he created functionality to remove bad data, as well as creating visualizations to support discovery and results.

All members of the group collaborated closely in creating the final report.

References

[1] *Clone Hero*. (n.d.). Clonehero.net. https://clonehero.net/

[2] Waissbluth, E., Carr, S., Popescu, A., & Hu, J. (2022). *Tensor Hero: Generating Playable Guitar Hero Charts from Any Song.* https://www.ischool.berkeley.edu/sites/default/files/sproject_attachments/tensorhero_capstone.pdf

[3] Chen, Y.-H., Huang, Y.-H., Hsiao, W.-Y., & Yang, Y.-H. (2020). *Automatic Composition of Guitar Tabs by Transformers and Groove Modeling.* ArXiv.org. https://arxiv.org/abs/2008.01431

[4] Kim, K., Yun, W., & Kim, R. (2015). *Clustering Music by Genres Using Supervised and Unsupervised Algorithms.* https://cs229.stanford.edu/proj2015/129_report.pdf

[5] Rouard, S., Massa, F., & Défossez, A. (2023). Hybrid transformers for music source separation. *ICASSP 2023 - IEEE International Conference on Acoustics, Speech and Signal Processing*.

[6] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[7] Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2024). RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 127063.

[8] Dao, T. (2023). FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*.

[9] Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. *International Conference on Learning Representations*.

[10] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations*.

[11] Park, D. S., Chan, W., Zhang, Y., Chiu, C. C., Zoph, B., Cubuk, E. D., & Le, Q. V. (2019). SpecAugment: A simple data augmentation method for automatic speech recognition. *Interspeech 2019*.

[12] genre. 2026. In Merriam-Webster.com. Retrieved March 3, 2026, from https://www.merriam-webster.com/dictionary/genre

[13] Grohotolski, M. (2026) feature_extraction.md. https://github.com/MattGroho/tab-hero/blob/main/docs/feature_extraction.md

Appendix

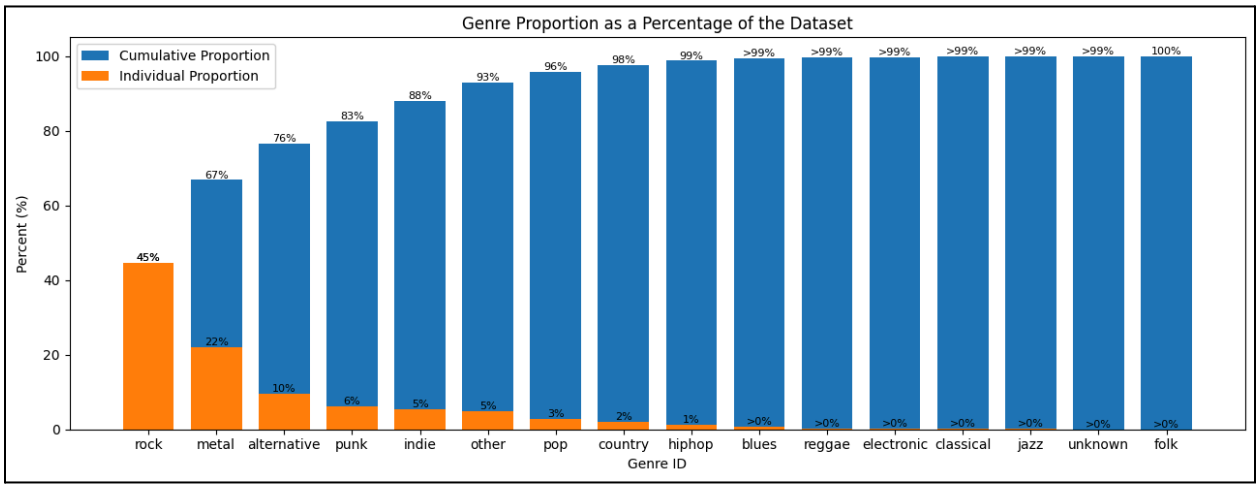

Figure 10: Cumulative and individual genre distribution present in the dataset. The dataset was heavily imbalanced, and a significant portion of the dataset consisted of Rock and Metal music.

Exploratory Data Analysis of the full feature set:
https://github.com/MattGroho/tab-hero/blob/main/notebooks/feature_exploration.ipynb

Exploratory Data Analysis of the cleaned feature set:
https://github.com/MattGroho/tab-hero/blob/main/notebooks/cleaned_rows_feature_exploration.ipynb